

Problem A. Convenient Coins

Idea: Ivan Androsov, preparation: Maksim Mescheryakov

First, let's try to understand how to determine the convenience of a multiset of coins. If the multiset does not contain any coins of value 1, then we definitely cannot make the sum 1, so the convenience is equal to 0.

On the other hand, if the multiset contains at least one coin of value 1, we can always obtain any sum from 0 to the total value: if we have obtained the sum x and it is not equal to the maximum possible value, then either it does not include one of the coins of value 1 (and we can add it), or it does not include one of the coins of value 2 (and we can replace a coin of value 1 with a coin of value 2). This way, we can get the sum $x + 1$.

Therefore, for each segment that contains at least one coin of value 1, the answer is equal to the sum on that segment. For other segments, the answer is equal to 0.

To obtain the sum of such values over all subsegments, we can use prefix sums to find the actual sum over all subsegments, and then subtract from it the total length of all subsegments consisting only of coins of denomination 2 (multiplied by 2). This way, we can achieve a solution in $O(n)$.

Problem B. Shortest Path

Idea and preparation: Adilbek Dalabaev

We define the degree of a black cell as the number of its black neighbors. Therefore, the degree of each black cell does not exceed 4.

Let's consider the path that a token will take — it will stop in some cell. Let's look at this cell — what was its degree at the beginning?

Case 1: The degree is 0. In this case, the answer is 1: the token started and ended in it. In other words, if there is an “isolated” cell on the field, it is optimal to start and end in it.

Case 2: The degree is 1. In this case, we can achieve an answer 2: we start in a neighboring cell and move to the chosen cell. In other words, if there is no case 1 on the field, but there is a cell of degree 1, then it will provide the best answer.

Case 3: The degree is 2. In this case, the optimal path looks as follows: we start in one neighbor u_1 of our vertex v and finish by entering v from another neighbor u_2 . It doesn't make sense to start anywhere but in a neighbor of v , since you have to visit both neighbors anyway to stop in v .

Thus, we need to find the shortest path from u_1 to u_2 for each cell v of degree 2. For this, it is sufficient to run bfs from one of the neighbors, prohibiting visits to vertex v . This part will work in $O((nm)^2)$ time.

Case 4: The degree is 3 (the weirdest case). Similar to the previous case, we start in one neighbor u_1 , then pass through the second neighbor u_2 , and finish in v , entering it from the third neighbor u_3 . However, note the following fact: every vertex along the entire path must have a degree of at least 3. Otherwise (if there is a vertex of degree 2 on the path), we can obtain a more optimal answer from the previous case.

Indeed, without loss of generality, let's assume that the vertex t of degree 2 lies on the path between u_1 and u_2 , then we can construct a cycle of the form $t \rightarrow u_2 \rightarrow v \rightarrow u_1 \rightarrow t$, where t fits case 3 and the answer is not greater than the current one.

In general, if the optimal answer indeed falls under case 4, we will obtain a very specific construction of the form:

?? ??
?1v3?
?y2??
?????

where v is the current vertex; 1, 2, 3 are its neighbors, also of degree at least 3, and “??” are cells of

unknown color. And since cell 2 must have at least 2 neighbors, at least one cell to the left or right of 2 must be black. Without loss of generality, let's assume this cell is y .

In other words, in the optimal answer, vertices 1 and 2 have a common neighbor. It can be proven (proof 1) that it is always optimal to connect 1 and 2 through y , while 3 can be connected via the shortest path either to 1 or to 2.

Furthermore, it turns out that an even stronger statement is true: if the answer falls under case 4, then it does not exceed 10. Specifically, only 2 constructions of the following forms are possible (up to rotation and reflection):

```
?? . ??
?XXX?
?XXX?
?????
```

The answer is 6.

```
?? . ??
?XXXX
?XX . X
??XXX
```

The answer is 10. See proof 2.

Case 5: The degree is 4. It can be proven that it is always possible to find an answer that is not worse, that falls under the previous cases (proof 3).

Thus, the solution consists of no more than nm runs of bfs, and therefore it works in $O((nm)^2)$ per test.

=== Proofs ===

Proof 1 (by contradiction): suppose 1 and 2 are not connected through y in the optimal answer. If there is some direct path between them, then by replacing it with $1 - y - 2$, we can only decrease the answer (we can do this since y will definitely be empty). Thus, we connected 1 with 3 and 2 with 3. If neither of these paths goes through y , we can replace the path from 1 to 3 with $1 - y - 2$ and obtain a shorter answer. If, without loss of generality, the path from 1 to 3 goes through y , we can replace it with the path $1 - y - 2$, and the answer will also decrease. In any case, we obtain a contradiction.

Let's introduce some constructions.

Construction 1 (up to reflections and rotations):

```
? . ??
.X??
?XX?
????
```

If this construction is present on the field, then the answer ≤ 4 . Proof: if $(2, 3)$ is empty, then the answer is 2, otherwise the answer is 4.

Construction 2 (up to reflections and rotations):

```
?????
?XXX?
?XXX?
?????
```

If this construction is present on the field, then the answer ≤ 6 . Proof: we find such a block that is located as far up and to the left as possible (regardless of whether it is 2×3 or 3×2). If cell $(1, 3)$ or $(4, 3)$ is empty (or does not exist), then we get answer 6, finishing at $(2, 3)$ or $(3, 3)$ respectively.

Otherwise, we get

```
??X??
?XXX?
?XXX?
??X??
```

If cell $(1, 2)$ is not empty, then we found a rectangle 2×3 above, meaning cell $(1, 2)$ is empty. If cell $(2, 1)$ is empty, then we found construction 1 and the answer ≤ 4 . Otherwise, cell $(2, 1)$ is not empty. If cell $(3, 1)$ is not empty, then we found 2×3 to the left, otherwise it is empty.

```
? .X??
XXXX?
.XXX?
??X??
```

Finally, if $(4, 2)$ is empty, we have construction 1, otherwise the answer is 6, if we finish at $(3, 2)$.

Construction 3:

```
? . .?
?33?
????
```

That is, a 2×2 square where two vertices are empty and the other two have degree 3. In this case, the answer is also no more than 6. Proof: we complete the mandatory cells

```
?? . .??
?XXXX?
??XX??
??????
```

If $(3, 2)$ or $(3, 5)$ are not empty, we obtain construction 2. Therefore, both must be empty:

```
?? . .??
?XXXX?
?.XX.?
??????
```

If either $(4, 3)$ or $(4, 4)$ is empty, we obtain construction 1. If both are not empty, then we have construction 2.

Proof 2. Recall that our path must pass only through vertices of degree at least 3 (otherwise case 3 is more optimal), and there should be no vertices of degree ≤ 1 on the field (since cases 1 and 2). We need to prove that if case 4 is optimal, then the answer does not exceed 10.

We have the following construction (vertices of degree ≥ 3 are underlined):

```
?? . ??
?XXX?
?XX??
?????
```

We know that one part of the path is $(2, 2) - (3, 2) - (3, 3)$, and the other part is some path from $(2, 4)$ either to $(2, 2)$ or to $(3, 3)$.

If $(3, 4)$ is not empty, then the answer is obviously 6. Otherwise:

1. $(3, 4)$ is empty;
2. $(4, 3)$ is not empty (since $(3, 3)$ has degree 3);
3. $(4, 2)$ is empty (otherwise, we have construction 2);
4. $(3, 1)$ is not empty (otherwise, we have construction 1);
5. $(2, 1)$ is empty (otherwise, we have construction 2);
6. $(1, 2)$ is not empty (otherwise, we have construction 1);
7. $(1, 4)$ is not empty (since $(2, 4)$ has degree 3);
8. $(2, 5)$ is not empty (since $(2, 4)$ has degree 3).

Or:

```
?X.X??  
.XXXX?  
XXX.??  
?.X???  
??????
```

Suppose we connect $(2, 4)$ with $(3, 3)$. The only way to reach $(3, 3)$ is through $(4, 3)$, meaning it also has degree 3. Therefore, $(4, 4)$ and $(5, 3)$ are not empty.

```
?X.X??  
.XXXX?  
XXX.??  
?.XX??  
??X???
```

Let's consider cell $(4, 5)$. If it is empty, we have construction 1.

Thus, let's consider the case where $(4, 5)$ is not empty:

```
?X.X??  
.XXXX?  
XXX.??  
?.XXX?  
??X???
```

Let's consider cell $(3, 5)$. If it is not empty, we will get an answer of 10. Otherwise, it is empty:

```
?X.X??  
.XXXX?  
XXX..?  
?.XXX?  
??X???
```

Now we need to look at cell $(2, 5)$. If it has degree 3, it will give us construction 3. If it has degree 2, then:

- either $(2, 6)$ is empty and we get construction 1,

- or $(1, 5)$ is empty. But then both $(1, 4)$ and $(2, 5)$ have degree no more than 2. However, the path exiting from $(2, 4)$ (and going to $(3, 3)$) must pass through one of them — this contradicts the fact that all vertices on the path have degree 3 or more.

Finally, we need to prove that if we connect $(2, 4)$ with $(2, 2)$, we will also get an answer no more than 10. We will add an additional row above our state (that is, $(2, 4)$ will turn into $(3, 4)$, $(2, 2)$ into $(3, 2)$, and so on):

```
??????
?X.X??
.XXXX?
XXX.??
?.X???
??????
```

Since we can only reach $(3, 2)$ from $(2, 2)$, then $(2, 2)$ has degree 3.

```
?X????
XX.X??
.XXXX?
XXX.??
?.X???
??????
```

If $(1, 3)$ is empty, we need to consider the degree of vertex $(1, 2)$ and, similarly, we will get:

- degree 3 leads to construction 3;
- degree 2 leads either to construction 1 or to a vertex of degree 2 on the path from $(3, 2)$ to $(3, 4)$.

Thus, $(1, 3)$ is not empty.

```
?XX???
XX.X??
.XXXX?
XXX.??
?.X???
??????
```

Finally, if $(1, 4)$ is empty, we have construction 1 (for $(2, 4)$), otherwise the answer is 10.

Proof 3: Similarly, if there is a vertex of degree 2 on the path, then there exists a shorter answer from case 3. Therefore, all neighbors of the vertex of degree 4 must have degree at least 3. We will prove that if there exists an answer with degree 4, then there also exists a shorter answer.

Let's start with the construction:

```
????
??X?
?XXX
??X?
```

Let's look at $(3, 2)$. If both $(2, 2)$ and $(4, 2)$ are not empty, we have construction 2.

Now consider the case where $(4, 2)$ is empty (the case of empty $(2, 2)$ is completely analogous). Then $(3, 1)$ and $(2, 2)$ are not empty:

```

????
?XX?
XXXX
?.X?

```

If either (2, 1) or (2, 4) is not empty, we have construction 2. Therefore, both must be empty:

```

????
.XX.
XXXX
?.X?

```

If

- (1, 2) is empty, we have construction 1,
- (1, 3) is empty, we have construction 1,
- both are not empty, we have construction 2.

Thus, if there exists case 5 (for which the answer is not less than 8), then there exists an answer not greater than 6.

Problem C. Paths in Matrix

Idea and preparation: Roman Glazov

Let's use the idea of contribution to the sum. For each value x , count the number of paths that pass through at least one cell with value x , and add this number times x to the answer.

There are two approaches one can follow.

Approach 1. Count the opposite: the number of paths that don't go through any cell with value x . Let $dp[i][j]$ be the number of such paths that end in a cell (i, j) . This dynamic programming can be calculated in $O(nm)$. Then the number of good paths is $\binom{n-1+m-1}{n-1} - dp[n-1][m-1]$.

Approach 2. Do a sparse dp only over cells with value x . Let $p[i]$ be the i -th such cell sorted in the increasing order of rows (and columns in case of equality). Let $dp[i]$ be the number of paths that end in $(p[i]_x, p[i]_y)$ and don't go through any $(p[j]_x, p[j]_y)$ for $j < i$. Basically, $dp[i]$ counts the number of paths such that $p[i]$ is the first cell with value x on the path. This can be calculated with a simple PIE.

$$dp[i] = \binom{p[i]_x + p[i]_y}{p[i]_x} - \sum_{j=0; p[j]_y < p[i]_y}^{i-1} dp[j] \cdot \binom{p[i]_x - p[j]_x + p[i]_y - p[j]_y}{p[i]_x - p[j]_x}.$$
 To find the total number of good paths, sum up $dp[i] \cdot \binom{n-1-p[i]_x+m-1-p[i]_y}{n-1-p[i]_x}$. So, once the path passes through the first cell with value x , it can proceed in any way. Thus, the solution takes $O(k^2)$, where k is the number of cells with value x .

Both solutions are rather slow on their own. Let's combine them. If k^2 is low, run the second approach. Otherwise, run the first approach. Naturally, the cut-off should be $\sqrt{nm}$. Then there are at most $\frac{nm}{\sqrt{nm}}$ runs of the first approach, that makes $O(nm\sqrt{nm})$ in total. Finally, the total complexity of the runs of the second approach is $O(\sum_x k_x^2) \leq O(\sqrt{nm} \cdot \sum_x k_x) = O(nm\sqrt{nm})$. So the solution runs in $O(nm\sqrt{nm})$ in total.

Problem D. Maximum Beauty Array

Idea: Ivan Androsov, preparation: Maksim Mescheryakov

Let's learn how to calculate the minimum cost to reduce the array to size 1. Fix the element at index i that remains that final one. We have to remove everything to the left and to the right of it. In order to

remove $i + 1$, we have to use the operation to remove the suffix from i . Any other operation would either not remove $i + 1$ or remove i together with it. At the same time, this operation removes the entire suffix, so it makes no sense to perform more suffix removals. Apply the same argument to $i - 1$ and prefixes. Thus, the array can be cleared in at most two operations, and the cost is at most $2 \cdot a_i$. Note that the cost might be smaller if i is 1 or n , since we don't have to remove the prefix or the suffix in these cases. For these, the answer is just a_i .

Thus, the minimum cost is $\min(\min(a_1, a_n), 2 \cdot \min_{i=2}^{n-1} a_i)$.

Since we want to maximize the cost, we'd like to put the minimum of the entire array somewhere from 2 to $n - 1$ and two large enough values at the sides. Note that the answer can't exceed $2 \min a$. So, there are two major cases.

There are at least two values that are not smaller than $2 \min a$. Then it's enough to move any two of them to the sides. The minimum won't be on a side, so the answer will indeed be $2 \min a$. To obtain the smallest number of swaps to achieve such configuration, we can move the leftmost such element to the left side and the rightmost such element to the right side.

All elements are smaller than $2 \min a$. Then the answer can't be smaller than the second maximum of the array. There will be some two elements on the sides, and the best way to maximize the minimum of them is to put two maximum values in the entire array on the sides. The idea for finding the smallest number of swaps is similar, though. We can find the leftmost and the rightmost elements that have values not smaller than the second maximum (note that equal elements might make it so there are more than 2 of them) and push them to the sides.

Overall complexity: $O(n)$.

Problem E. Buy for More, Sell for Less

Idea and preparation: Ivan Androsov

First, let's solve the problem in $O(n)$ per query, and then we will try to speed it up.

It is quite obvious that as soon as we buy an item, we can immediately sell it. Therefore, each type of item defines the operation "if we have at least b_i coins, we can obtain one unit of joy for the price $d_i = b_i - s_i$ ".

We can sort all types of items in non-decreasing order of b_i , then for each current amount of coins k , some prefix of item types will be available to us. It is easy to see that from this prefix, it is optimal to choose the minimum value of d_i .

Let's sort not just by non-decreasing b_i , but also, in case of equality, by non-increasing d_i , and eliminate "redundant" types of items (we will consider types with d_i not less than the minimum previous value in the sorted order as "redundant"). We will end up with a filtered list of items where b_i strictly increases and d_i strictly decreases. Now it is always optimal to use the last item available to us. Let's say we now have a list of pairs (b'_i, d'_i) of length m after sorting.

We can design a solution in $O(m)$ per query: we will try to buy and sell the last item as many times as possible, then do the same with the second-to-last item, and so on, until we find that we do not have enough money even for the first type of item.

Unfortunately, this is too slow. To optimize it, we will formulate a solution using the following dynamic programming: let dp_k be the amount of joy we will get if we initially have k coins; then:

- $dp_k = 0$ for $k < b'_1$;
- $dp_k = dp_{k-x} + 1$, where $x = d'_i$ for the maximum b'_i not exceeding k .

We can speed up this dynamic programming in two ways: the method from the author's solution and a significantly smarter one. Both methods are based on the fact that we have m "segments" of values, each with its own "period" d'_i , and within this segment, each subsequent block of length d'_i is the previous block of the same length with all values increased by 1.

Author's solution (works quite slowly and requires a lot of memory, not very easy to implement):

Let's store the values of dynamic programming in a persistent implicit-key treap. We will eliminate priorities in the treap tree to be able to call merge with itself; instead of priorities, we will use size-based balancing (if we merge two trees of sizes x and y , the root of the left one becomes the root of the merged tree with a probability of $\frac{x}{x+y}$). We will also implement a lazy addition operation on the entire subtree.

How will this help us? To calculate the values over some segment with a period, we need to take the last d'_i values from the previous segment and copy them many times, adding 1 to the right part with each copy. This can be done in $O(\log M)$ merge calls if you use something like binary exponentiation (use binary exponentiation, but instead of multiplication, add some value to the second tree, then merge them).

Thus, each segment will be built in $O(\log^2 M)$, and we will get a tree of depth $O(\log M)$. By descending down this tree, we can obtain the answer to the query. We get a solution which works in $O(n \log^2 M + q \log M)$.

Unfortunately, this is quite a heavy solution, especially in terms of memory usage. It is likely that it will not pass without some optimizations, for example:

- in each node of the treap, do not store the value and the lazy addition separately, but combine them together and don't "push" the lazy addition to children (you will have to recalculate these values for nodes whose ancestors change, but this will reduce both the number of node copies and the size of a single node);
- use `int` instead of pointers to the left/right child (this will reduce the pointer size by half).

Smart solution (thanks to Dominater069 for it, we did not come up with this ourselves):

We will use the fact that for each value k , it holds that $dp_k \in [dp_{k-1}, dp_{k-1} + 1]$.

For each segment, we will calculate two values:

- $a_i = dp_{b'_i}$ (the value of the dynamic programming at the starting point of the segment);
- c_i — the minimum j for which $dp_{c_i} > a_i$ (the first point where the value will be greater than at the start).

It turns out that this is enough to calculate the dynamic programming value at any point in this segment:

- over the entire interval $[b'_i, b'_i + d'_i - 1]$, the values can be calculated as follows: if $k < c_i$, then $dp_k = a_i$, otherwise $dp_k = a_i + 1$;
- and then this interval repeats with an increase of 1 with each repetition.

The second of these statements is obvious because the length of the interval equals the "period" of the chosen segment. Let's prove the first one. Suppose there exists some $x \in [b'_i, b'_i + d'_i - 1]$ such that $dp_x > dp_{b'_i} + 1$. Due to the fact that dp is non-decreasing, this means that $x > c_i$. Consider $dp_{x-d'_i}$:

- on one hand, it must be greater than a_i for dp_x to be greater than $a_i + 1$;
- on the other hand, $x - d'_i < c_i$, because x and c_i are in the same segment of d'_i values; and since c_i is the first point where the value is greater than a_i , it follows that $dp_{x-d'_i} < a_i + 1$.

We arrive at a contradiction, so such a situation is impossible.

This allows us to calculate any value of dp_k in $O(\log n)$, simply knowing which segment this point is in (the segment can be found using binary search). The values a_i and c_i for all segments are calculated as

follows: $a_i = dp_{b'_i - d'_i} + 1$ (which we already know how to calculate); c_i can be found using binary search due to the monotonicity of dp (but can also be done faster).

Thus, we obtain a solution in $O(n \log n \log M + q \log n)$ or $O(n \log n + q \log n)$ depending on the implementation.

Problem F. Graph Stabilization

Idea and preparation: Ivan Androsov

If we remove edges from a graph, its stability can only decrease. Therefore, we can calculate the stability of the original graph, remember which vertices were not removed, and assume that none of these vertices should be removed during stabilization after we erase some edges of the graph. We can even immediately remove all vertices that will never remain after stabilization, along with their incident edges, after which we solve the problem where no vertex should become unstable.

If we keep an edge (i, j) such that $a_i < a_j$, and vertex i must ultimately be stable, then vertex j will also be stable. We say that in this case, this edge *stabilizes* vertex j . Moreover, if $a_i \oplus a_j = k$, then this edge also stabilizes vertex i . Our task is the following: for each vertex, leave at least one edge which stabilizes it.

We will divide the edges into two sets: those that stabilize only one vertex and those that stabilize both ends (if an edge does not stabilize either end, it can be removed immediately — this happens when $a_i = a_j$). We will call the edges of the first type *soft edges*, and the edges of the second type *hard edges*.

For each vertex, we need at most one soft edge that stabilizes it (or none at all if we stabilize it with a hard edge). Let the minimum soft edge that stabilizes vertex i have a cost of b_i (or $b_i = \infty$ if such an edge does not exist). We can then reformulate the problem as follows: given a set of vertices, each of which has to be stabilized; each vertex can be stabilized separately for b_i ; there are also several edges (the hard edges of the original graph) with their own costs that stabilize both ends.

We can almost solve such a problem. The only observation left is the following: since $k > 0$, any cycle of hard edges will have an even length. This means that the graph consisting of hard edges is bipartite. For example, we can split it into two parts as follows:

- vertices for which $a_i \oplus k > a_i$;
- vertices for which $a_i \oplus k < a_i$.

This allows us to solve the problem using minimum cost flows. Let's create a flow network where for each vertex in the first part, there is a directed edge from s to it, and for each vertex in the second part, there is a directed edge from it to t . We say that a vertex is stabilized if and only if at least one unit of flow passes through this edge — that is, we need to set a lower bound on the flow for this edge equal to 1 (the upper bound on the flow will be n , as we can potentially take all hard edges incident to some vertex). For each hard edge, we add the possibility of stabilizing its ends at a cost of c_i — that is, we add a directed edge from the left vertex to the right vertex with a capacity of 1 and a cost of c_i . For each vertex in the left part, we add an edge from it to the sink with a capacity of 1 and a cost equal to b_i (to allow stabilization of the vertex with a soft edge). Finally, for each vertex in the right part, we add an edge from the source to it with a cost of b_i .

In this network, we need to find a minimum-cost flow that satisfies all lower bounds. The lower bounds can be handled in two ways — either by using a dummy source/sink or by splitting an edge with a lower bound L and an upper bound R into two:

- with a capacity of $R - L$ and a cost equal to the cost of the original edge;
- and with a capacity of L and a cost of $-\infty$.

The minimum cost flow in such a network will give us the set of edges that need to be kept in the graph while minimizing the sum of c_i . At first glance, this solution works in $O(n^5)$: the flow value can be up to $O(n^2)$, and one augmenting path is found in $O(n^3)$. However, it actually works in $O(n^4)$, as once the flow value reaches n , the subsequent augmenting paths are no longer needed (the previous paths have saturated all edges that need to be saturated).

This solution can be improved to $O(n^3)$ by finding the augmenting path using Dijkstra's algorithm with Johnson's potentials or reformulating the problem in the format of an assignment problem instead of a minimum cost flow, but this was not required.

Problem G. Find a Better String

Idea and preparation: Ivan Androsov

Let's reinterpret the binary strings into something that's easier to reason about. In particular, let zeroes in the binary string form a first set of values and ones form a second set of values. Then we look for unique sums of one value from one set and another value from another set.

Consider the worst case: the hidden string has the maximum possible cost. To solve this test, we'll have to check if the hidden string is the maximum one in only three queries. Since three queries is not a lot, such string might look a very specific way.

Thus, let's analyze what a maximum cost string might look like.

In theory, one might obtain all sums from 3 to $2n-1$. In practice, it isn't quite achievable. The construction would go as follows:

- 1 belongs to the first set;
- in order to obtain sum 3, we have to put 2 into the second set;
- in order to obtain sum 4, we have to put 3 into the second set;
- ...
- in order to obtain sum n , we have to put $n-1$ into the second set.

However, if we proceed with that construction to get sum $n+1$ and put n into the second set, these will be all the sums. Nothing larger than $n+1$ will be obtained. Thus, at least one of the sums from 3 and $2n-1$ can't be obtained.

If we stop the construction here, the sets will look like this: the values 1 and n are in the first set, the rest are in the second set. Thus, all sums from 3 to n and from $n+2$ to $2n-1$ can be obtained.

Let's show that no other construction obtains the same number of unique sums. Let the smallest sum that can't be obtained be some l . That means that 1 and $l-1$ belong to the same set (from the construction above). Similarly, the largest sum $n+r$ that can't be obtained forces some $r+1$ and n to belong to the same set (the same construction can be redone from the largest to the smallest sums). If 1 and n belong to the same set and the rest belongs to another set, then $l=r$. Otherwise, at least two sums can't be obtained, since either the sum that comes from l is smaller than $n+1$ or the sum that comes from r is greater than $n+1$.

That argument has a weird corner case: there are no elements in the set with 1 or in the set with n . Then only $n-1$ sums can be obtained. And that is actually optimal for $n=3$.

To sum that up, the maximum cost string looks as follows:

- for $n=3$, the string should have at least one 0 and at least one 1;
- for $n>3$, $s_1=s_n$ and $s_i \neq s_1$ for all i from 2 to $n-1$.

Well, since the construction is that simple, we can just check if the hidden string is exactly that, and print it if it isn't.

For example, for $n = 3$, one can query $[1, n]$ and deduce the answer from it. If it's 2, then it's maximum; otherwise, it isn't. For $n > 3$, one can query $[2, n - 1]$, $[2, n]$ and $[1, n - 1]$. If the answers are 1, 2, 2, then it's maximum; otherwise, it isn't.

Problem H. Inverted Subarray

Idea: Ivan Androsov, preparation: Maksim Mescheryakov

Let's find the elements that don't belong to any inversion in the entire array. So, they are larger than all elements to the left and smaller than all elements to the right. Obviously, no inverted subarray contains any of them. If they don't belong to an inversion at all, they will still not belong to one after removing some elements.

Then the claim is that if you take all elements between two such bad ones, that subarray will be a valid inverted subarray.

Let's show that by contradiction. Take an element i inside this subarray. Let $l - 1$ and $r + 1$ be the bad elements outside the subarray. Let i belong to an inversion in the full array but not in the subarray. Let there be a larger element j to the left of it outside the subarray. Since the bad element $l - 1$ doesn't form an inversion with i , $a_{l-1} < a_i$. But that means the $a_{l-1} < a_j$, which is to the left of it. Thus, $l - 1$ belongs to an inversion, which is a contradiction. Similar analysis can be performed for $r + 1$.

Thus, the solution is to find all bad elements and print the longest segment that doesn't contain any of them. That can be done with two pointers.

Overall complexity: $O(n)$.

Problem I. Red-Black Graph

Idea: Ivan Androsov, preparation: Maksim Mescheryakov

First, let's try to find the maximum number of edges in a beautiful graph with n vertices. Let there be r edges with both ends red, b edges with both ends black, and k edges with different ends. Then the total number of edges m is equal to $r + b + k$.

According to the problem statement, $r + k = n - 1$, and also $b + k = n - 1$ (since these sets of edges must form trees). This means that $b = r$, and thus $m = 2 \cdot (n - 1 - k) + k$. Therefore, the smaller k is, the more edges there are in the graph. We would like to minimize the number of edges connecting vertices of different colors.

On the other hand, if any red vertex is not connected to a black vertex, it will be isolated if all red edges are removed. The same applies to each black vertex. Therefore, each vertex must be incident to at least one edge connecting it to another color. This means that $2k \geq n$, so the minimum value of k that is available to us is $\lceil \frac{n}{2} \rceil$. This estimate is always achievable, and we will show this constructively during the graph counting process.

Suppose n is even, then $n = 2k$. Since exactly k edges must connect vertices of different colors, both the number of red vertices and the number of black vertices must equal k . The edges with different colored ends then form a perfect matching, and the number of such matchings is equal to $k!$. The edges between red vertices must form a tree (otherwise, either cycles of "red" edges will appear, or the graph will be disconnected without "black" edges), and the number of such trees is equal to k^{k-2} (as follows from Prüfer's code). The same applies to the edges between black vertices. And we also need to choose which vertices are red and which are black. Therefore, the answer for even n is $(k^{k-2})^2 \cdot k! \cdot \frac{n!}{(k!)^2}$.

For odd n , one of the vertices will be connected to two vertices of the other color, and the number of black vertices will differ from the number of red vertices by 1. Let's assume that the number of black vertices is k , and the number of red vertices is $k - 1$ (the number of graphs where this is reversed will be equal to the number of graphs where it is as stated).

One of the red vertices will be connected to two black vertices. Let's choose this red vertex and this pair of black vertices; there will be $\frac{k \cdot (k-1)^2}{2}$ ways to do this. The remaining edges between red and black vertices must form a perfect matching, so the number of ways to choose them is $(k-2)!$.

The edges between red vertices must form a tree, so the number of ways to choose them is $(k-1)^{(k-3)}$. The situation with the edges between black vertices is a bit more complicated: two black vertices are already connected, so we need to count the number of ways to construct a tree with one fixed edge. Let's assume that this edge is removed from the resulting tree, and let's see how many vertices will be in each resulting component; let this be i and $(k-i)$. Each of the resulting components will be a tree, so the number of ways to choose these two trees will be $i^{(i-2)} \cdot (k-i)^{(k-i-2)}$ for a fixed i , and we just need to sum over i to get the number of ways to connect the black vertices.

Thus, we can analyze the case of odd n in $O(n)$. A faster formula can be obtained, but this was not required in the problem.

Problem J. Disks and Gems

Idea and preparation: Ivan Androsov

The solution to this problem consists of two steps:

- derive a formula for the answer;
- compute it quickly.

Deriving the formula for the answer is not very difficult. We need to count the number of distinct ornaments m , and then raise 3 to the power of m . For each ornament, we make a choice k times: first, we choose one of n sectors for type 1, then one of $n-2$ sectors for type 2, and so on. If we make different choices anywhere, we get different ornaments, except for one case: when there are only two sectors left and both need to be decorated with the same type of gem, it does not matter which one we choose. This can be handled by reducing k by 1 in advance if $2k = n$.

Thus, we obtain the following formula for the answer: $3^m \bmod 998244353$, where

$$m = n \cdot (n-2) \cdot (n-4) \cdots (n-2k+2)$$

Now we need to understand how to calculate this quickly. We will use the fact that $p = 998244353$ is the modulus for FFT, which means it is a prime number such that $p-1$ is divisible by a sufficiently large power of two. To be more precise, $p = 2^{23} \cdot 7 \cdot 17 + 1$.

Since p is a prime number, $\varphi(p) = p-1$. Moreover, since $\gcd(3, p) = 1$, we have $3^{\varphi(p)} \bmod p = 1$. This means that we only need to calculate m modulo $p-1$.

However, computing modulo $p-1$ is still not easy; instead, let's compute separately:

- $m \bmod 7$;
- $m \bmod 17$;
- $m \bmod 2^{23}$.

Then, from these three values, we can reconstruct $m \bmod (p-1)$ using the Chinese remainder theorem.

Now we need to understand how to compute m modulo these three numbers. The model solution does this as follows: for each of the three moduli, we precompute $x!!$ (the double factorial) for each x from 0 to double the modulus in the format of a pair (y, z) , where:

- y is the maximum power of some prime (2, 7, 17 for the moduli $2^{23}, 7, 17$, respectively) that divides $x!!$;

- z is the value of $x!!$ modulo the required modulus, excluding that prime.

From such precomputation, it is easy to obtain the value of $x!!$ for any x in $O(\log x)$ time.

This allows us to compute m as $\frac{n!!}{(n-2k)!!}$ and use “division” in this formula.

This solution can be simplified as follows: for sufficiently large k , the values $m \bmod 7$ and $m \bmod 17$ are both 0, which allows us to precompute only for the modulus 2^{23} , and for small k , we can compute $\prod_{i=0}^{k-1} 3^{n-2i}$ in $O(k \log n)$ time.

Problem K. Three Hundred Queries

Idea and preparation: Adilbek Dalabaev

Ask a random point x . Let the outcome be some distance d . That means that either $x + d$, or $x - d$ is one of the hidden points. Let's use another query to determine which of these two is actually a point. Ask a point $x + 1$. If the outcome is $d - 1$, then the point is $x + d$. Otherwise, the point is $x - d$. If you ask at that point, the outcome will be the smaller of the distances from it to both other hidden points. Save the result of the third query. Repeat 100 times.

At the end, there will be 100 values with each being one of the two shortest distances between points. If the two shortest distances are the same, then it's just 100 of the same value. Otherwise, it's two different values. The answer is obtained trivially from them.

Let's calculate the probability that this approach works.

First, we assumed that neither x , nor $x + 1$ lands in a hidden point. That can actually be fixed by not recording the value of the third query if one of the first queries two returned 1. That happens with probability $\frac{12}{10^9}$. That is negligible over all tests.

Second, we have to estimate how often we get each of the two smallest distances. Consider the largest arc on the circle. With probability at least $\frac{1}{3}$, the query lands inside this arc. Then, it lands on the left or on the right side of it with probability around $\frac{1}{2}$. And the arc of the result depends on this coin toss. Thus, each query returns one of the two smallest distances with probability at least $\frac{1}{6}$. So, the probability to never encounter an arc is $(\frac{5}{6})^{100}$, which is around 10^{-8} . Even assuming the problem has 100 tests, it's enough to pass comfortably.

Problem L. Distinct/Insertion Queries

Idea and preparation: Alexey Shnirelman

There is a well-known technique that allows us to solve this problem on a fixed array: for each element, we find the position of the previous occurrence of the same element (or -1 if there was none before). For the segment $[l, r]$, the elements whose previous occurrence is before l are the first occurrences in the segment, while the others are not first occurrences. This reduces the problem to the query “count of smaller elements in the segment”, which can be solved using a mergesort tree or a persistent segment tree.

Unfortunately, we cannot use this technique directly if we need to insert elements into the array (as inserting an element changes too many indices). However, we can do something similar.

We will use sqrt-decomposition: we will divide all elements into blocks of size approximately $\sqrt{n+q}$, each of which will support element insertion; if at any point one of the blocks becomes significantly larger than its original size (for example, twice as large), we will rebuild the whole decomposition from scratch. For each element, we will maintain **in which block** its previous occurrence is located.

If we need to answer the query “count the number of distinct elements in $[l, r]$ ”, let's divide the elements into three groups. Let k be the block number where the left boundary of the query lies, then:

- elements with previous occurrences to the left of k will definitely be the first occurrences;

- elements with previous occurrences to the right of k will definitely not be the first occurrences;
- elements with previous occurrences in block k may behave differently.

For each block that is completely within $[l, r]$, we can count the number of elements with previous occurrences to the left of block k in $O(1)$. We will then need to process the elements from the blocks where the left and right boundaries lie (since these blocks cannot be taken completely), as well as the elements whose previous occurrence is exactly in block k . However, the total number of these elements is $O(\sqrt{n+q})$, and among them, we can count all distinct elements in linear time (for this, we can maintain an array where for each number from 1 to n , we mark in which query we processed it last, allowing us to count each number only once in the query).

This is the general idea of the solution. Next are the implementation details. We would like to achieve an asymptotic complexity of $O((n+q)\sqrt{n+q})$; with some effort, it is possible to get Accepted with a solution with worse asymptotic complexity, but that is quite challenging.

Let B denote the number of blocks. In each block, we will store:

- an array of size $B + 1$, where the i -th number is the count of elements whose previous occurrence in the block is less than i (this will allow us to process the block in the query in $O(1)$);
- an array of vectors of size $B + 1$, where the i -th vector will store all elements whose previous occurrence is in block i (this will allow us to quickly iterate over all elements with a previous occurrence in a specific block);
- the elements themselves that belong to the block, along with the block numbers of their previous occurrences.

The query “count the number of distinct elements” is now processed trivially. When inserting an element, we need to find the previous and next occurrences of that element to understand which blocks need to be modified. For each element, we can maintain a list of blocks where it appears at least once; this will help find the nearest block to the left/right that contains the same element. However, it is not very convenient to handle the case where the nearest occurrence to the left/right may be in the same block; there are different ways to resolve this issue, the model solution stores **the previous occurrence in a different block** for each element, and this is easier to maintain.